

BT Wholesale — AEM Headless Content Fragment Architecture

BT Wholesale — AEM Headless Content Fragment Architecture

Technical Design Document

Project: BTWholesale AEM as a Cloud Service

AEM SDK Version: 2026.5.25892.20260506T135241Z-260400

Author: BT Wholesale Development Team

Status: Active

Last Updated: 2026-05-14

Table of Contents

1. [Executive Summary](#)
 2. [Architecture Overview](#)
 3. [Project Module Structure](#)
 4. [Content Fragment Model Architecture](#)
 - 4.1 [Model Node Structure](#)
 - 4.2 [Field Types Reference](#)
 - 4.3 [Model Inventory](#)
 5. [Content Fragment Data Architecture](#)
 - 5.1 [Fragment Node Structure](#)
 - 5.2 [Field Name Convention](#)
 - 5.3 [Fragment Inventory](#)
 6. [Headless API — Exposing Content](#)
 - 6.1 [AEM GraphQL API](#)
 - 6.2 [AEM Content Services \(JSON Exporter\)](#)
 - 6.3 [AEM Assets HTTP API](#)
 - 6.4 [Choosing the Right API](#)
 - 6.5 [GraphQL Endpoint Configuration](#)
 - 6.6 [GraphQL Type Names](#)
 - 6.7 [Example GraphQL Queries](#)
 - 6.8 [Persisted Queries](#)
 - 6.9 [Next.js / React Integration Pattern](#)
 7. [Inter-Fragment Reference Map](#)
 8. [Deployment](#)
 9. [Design Decisions and Constraints](#)
 10. [Folder Structure Reference](#)
-

1. Executive Summary

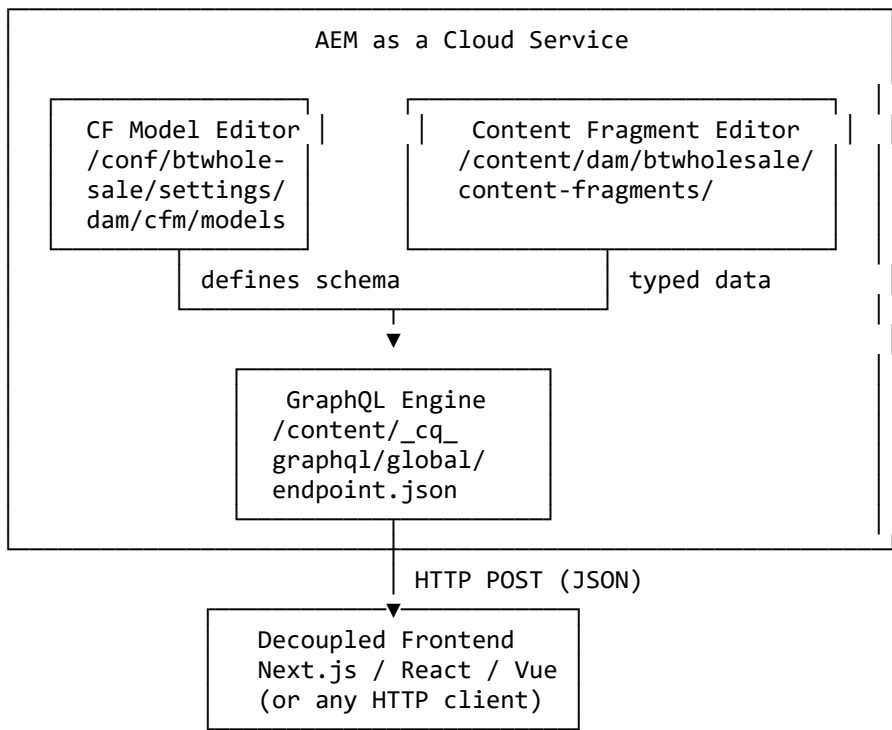
BT Wholesale’s content layer is built on **AEM as a Cloud Service** using the headless Content Fragments capability. All page content is structured as typed data in Content Fragments (CFs) — not in page templates or components — and consumed by a decoupled frontend via AEM’s **GraphQL API**.

The architecture covers three page scopes:

Scope	Page	Description
Acquisition	Become a Customer	Benefits, product catalogue, Why BT, apply journey
Support	Help & Support	Topic categories, contact methods, partner portals
Global	Site Content	Header nav, hero, footer links, testimonials, product cards

All content is model-driven: every authored value traces back to a typed CF model field, and every frontend value traces back to a GraphQL query on that model. There is no bespoke rendering in AEM — the frontend owns all layout and presentation.

2. Architecture Overview



Key principle: AEM is a pure content repository. The frontend fetches structured JSON; AEM never renders HTML for end-user pages.

3. Project Module Structure

```
btwholesale/
├── core/           Java bundles – OSGi services, Sling models, servlets
├── ui.apps/        AEM components, templates, clientlibs (/apps)
├── ui.content/     Content packages – CF models + CF data (this document)
│   ├── src/main/content/jcr_root/
│   └── conf/btwholesale/settings/dam/cfm/models/
```

└─ content/dam/btwholesale/content-fragments/	
└─ ui.config/	OSGi run-mode configurations
└─ ui.frontend/	Frontend build (Webpack/React/Angular)
└─ all/	Single deployable package
└─ pom.xml	

4. Content Fragment Model Architecture

4.1 Model Node Structure

Every CF model is a cq:Template node. The complete required structure is:

```
<jcr:root jcr:primaryType="cq:Template"
  allowedPaths="/content/entities(/.*)">

  <jcr:content
    cq:scaffolding="...jcr:content/model"
    cq:status="enabled"
    cq:templateType="/libs/settings/dam/cfm/model-types/fragment"
    jcr:primaryType="cq:PageContent"
    jcr:title="Model Display Title"
    sling:resourceSuperType="dam/cfm/models/console/components/data/entity">

    <model
      jcr:primaryType="cq:PageContent"
      sling:resourceType="wcm/scaffolding/components/scaffolding"
      dataTypesConfig="/mnt/overlay/settings/dam/cfm/models/formbuilderconfig/datatypes"
      maxGeneratedOrder="N">

      <cq:dialog sling:resourceType="cq/gui/components/authoring/dialog">
        <content sling:resourceType="granite/ui/components/coral/foundation/fixedcolumns">
          <items maxGeneratedOrder="N">
            <f0 ... />
            <f1 ... />
          </items>
        </content>
      </cq:dialog>
    </model>
  </jcr:content>
</jcr:root>
```

Critical: model node must have jcr:primaryType="cq:PageContent" and sling:resourceType="wcm/scaffolding/components/scaffolding". Any deviation causes a this.scaffold is null 500 error in the CF Model editor.

Console visibility: All models must be placed directly under /conf/btwholesale/settings/dam/cfm/models/. Sub-folders (e.g. shared/) are not traversed by the AEM CF Models console and the models will be invisible.

4.2 Field Types Reference

Type	sling:resourceType	metaType	Notes
Single-line text	granite/ui/components/coral/foundation/form/textfield	text-single	Add multiple="{Boolean}true" for string arrays

Type	sling:resourceType	metaType	Notes
Multi-line rich text	dam/cfm/admin/components/authoring/contenteditor/multieditor	text-multi	cfm-element must equal name; returns { html } in GraphQL
Fragment reference	dam/cfm/models/editor/components/fragmentreference	fragment-reference	Set fragmentmodelreference to model conf path
Multi fragment reference	same as above	fragment-reference	Add multiple="{Boolean}true"

Fragment reference required attributes:

```
<fn
  sling:resourceType="dam/cfm/models/editor/components/fragmentreference"
  fieldLabel="Label"
  filter="hierarchy"
  fragmentmodelreference="/conf/btwholesale/settings/dam/cfm/models/MODEL_NAME"
  metaType="fragment-reference"
  name="fieldName"
  nameSuffix="contentReference"
  rootPath="/content/dam/btwholesale"
  valueType="string/content-fragment">
  <field jcr:primaryType="nt:unstructured" rootPath="/content/dam/btwholesale"/>
  <granite:data jcr:primaryType="nt:unstructured"/>
</fn>
```

4.3 Model Inventory

All 10 models live flat under /conf/btwholesale/settings/dam/cfm/models/:

Component Models

Model	Fields
page-meta	title, description
page-hero	breadcrumb, breadcrumbHref, heading*, subheading (rich)
icon-card	id, icon, heading*, description (rich), highlight
nav-category	id, icon, heading*, description (rich), linkLabel, linkHref
link-item	label, href
contact-card	id, icon, heading*, description (rich), label, href
portal-item	id, icon, heading*, description (rich), label, href

* required field

Page Models

Model	Key Fragment-Reference Fields
become-a-customer	meta → page-meta, hero → page-hero, benefits[] → icon-card, products[] → nav-category, whyBT[] → icon-card, applyCta → link-item
help-content	meta → page-meta, hero → page-hero, categories[] → nav-category, contactCards[] → contact-card, viewAllContacts → link-item, portals[] → portal-item

Model	Key Fragment-Reference Fields
site-content	meta → page-meta, utilityLinks[] → link-item, navLinks[] → link-item, authLinks[] → link-item, productCards[] → icon-card, supportActions[] → link-item, whyCards[] → icon-card, footerLegalLinks[] → link-item, footerSocialLinks[] → link-item

5. Content Fragment Data Architecture

5.1 Fragment Node Structure

Every content fragment is a dam:Asset node under /content/dam/btwholesale/content-fragments/:

```
<?xml version="1.0" encoding="UTF-8"?>
<jcr:root xmlns:jcr="http://www.jcp.org/jcr/1.0"
  xmlns:dam="http://www.day.com/dam/1.0"
  xmlns:nt="http://www.jcp.org/jcr/nt/1.0"
  xmlns:cq="http://www.day.com/jcr/cq/1.0"
  jcr:primaryType="dam:Asset">

  <jcr:content
    cq:name="NODE_NAME"
    cq:parentPath="/content/dam/btwholesale/content-fragments/PARENT"
    jcr:description=""
    jcr:lastModified="{Date}2024-01-01T00:00:00.000-08:00"
    jcr:lastModifiedBy="admin"
    jcr:primaryType="dam:AssetContent"
    jcr:title="Fragment Title"
    contentFragment="{Boolean}true"
    lastFragmentSave="{Date}2024-01-01T00:00:00.000-08:00">

    <data
      cq:model="/conf/btwholesale/settings/dam/cfm/models/MODEL_NAME"
      jcr:primaryType="nt:unstructured">

      <master
        jcr:mixinTypes="[cq:Taggable,dam:cfVariationNode]"
        jcr:primaryType="nt:unstructured"
        fieldName="value"
        fieldName_x0040_LastModified="{Date}2024-01-01T00:00:00.000-08:00"/>
      </data>

      <metadata
        jcr:mixinTypes="[cq:Taggable]"
        jcr:primaryType="nt:unstructured"/>
      <related jcr:primaryType="nt:unstructured"/>
    </jcr:content>
  </jcr:root>
```

5.2 Field Name Convention

Pattern	Description
fieldName="value"	Plain string field
fieldName_x0040_LastModified="{Date}..."	Required timestamp after every field (_x0040_ = URL-encoded @)
fieldName="[v1,v2,v3]"	Multi-value string array
fieldName="/content/dam/btwholesale/..."	Fragment reference (single)

Pattern	Description
fieldName="[/content/dam/...,/content/dam/...]"	Multi fragment reference array

The model field name attribute **must exactly match** the JCR attribute name in master. A mismatch returns null in GraphQL with no error.

5.3 Fragment Inventory

content-fragments/	
— become-a-customer/	
— page	BecomeACustomerModel – top-level aggregator
— meta	PageMetaModel
— hero	PageHeroModel
— apply-cta	LinkItemModel
— benefits/	quality, network, revenue, support (IconCardModel × 4)
— products/	data, voice, hosted-comms, managed-services, mobile, iot (NavCategoryModel × 6)
— why-bt/	innovation, reach, bespoke (IconCardModel × 3)
— help-content/	
— page	HelpContentModel – top-level aggregator
— meta	PageMetaModel
— hero	PageHeroModel
— view-all-contacts	LinkItemModel
— categories/	broadband, ethernet, hosted-communications, product-documentation, customer-service-plans, pricing, billing, apps, network-information, regulatory, contracts, faqs, mobile (NavCategoryModel × 13)
— contact-cards/	enquiry, callback, phone (ContactCardModel × 3)
— portals/	my-bt-wholesale, the-hub (PortalItemModel × 2)
— site-content/	
— page	SiteContentModel – top-level aggregator
— meta	PageMetaModel
— footer/	partner-cta, legal-*, social-* (LinkItemModel × 7)
— links/	nav-*, auth-*, utility-*, hero-cta (LinkItemModel × 11)
— product-cards/	data-connectivity, hosted-communications, teams-phone-mobile, voice-services (IconCardModel × 4)
— support-actions/	call, contact, help (LinkItemModel × 3)
— why-cards/	expertise, innovation, reach (IconCardModel × 3)

6. Headless API — Exposing Content

AEM as a Cloud Service offers three distinct headless APIs. Each suits different use cases.

6.1 AEM GraphQL API

Best for: Page-level data fetching, nested fragment traversal, frontend applications.

The GraphQL API is auto-generated from CF model definitions. AEM introspects the models under /conf and creates a typed schema — no custom resolvers needed.

Capabilities: - Query a single fragment by path (byPath) - Query all fragments of a model type (List) - Filter, paginate, and sort results - Traverse nested fragment references in one request - Persisted queries (cacheable GET requests) - Rich text returns { html }, { plaintext }, { markdown } projections

Limitations: - Read-only (no mutations) - Schema driven by CF models — adding a field requires a model change + redeploy - No custom resolvers or computed fields

6.2 AEM Content Services (JSON Exporter)

Best for: Traditional AEM page components that need JSON output; experience fragments.

Append `.model.json` to any AEM page or component path:

GET `/content/btwholesale/us/en/home.model.json`

Returns a JSON representation of the page's component tree. Suitable when some content is still in AEM Pages (not purely headless).

6.3 AEM Assets HTTP API

Best for: DAM operations, programmatic CRUD on fragments, integration scripts.

GET `/api/assets/btwholesale/content-fragments/become-a-customer/page.json`

POST `/api/assets/btwholesale/content-fragments/` (create)

PUT `/api/assets/btwholesale/content-fragments/become-a-customer/page` (update)

Returns full `dam:Asset` metadata. Suitable for CI/CD pipelines, migration scripts, or CMS integrations that need write access.

6.4 Choosing the Right API

Need	Recommended API
Frontend page rendering	GraphQL
Multiple fragments in one request	GraphQL
Filtering / pagination	GraphQL
CDN-cacheable content delivery	GraphQL Persisted Queries
Experience Fragments / SPA Editor	Content Services (<code>.model.json</code>)
Programmatic CF CRUD	Assets HTTP API
Migration / bulk operations	Assets HTTP API

6.5 GraphQL Endpoint Configuration

Environment	Endpoint
Local Author (read/write)	<code>http://localhost:4502/content/_cq_graphql/global/endpoint.json</code>
Local GraphQL Explorer	<code>http://localhost:4502/content/_cq_graphql/global/explorer</code>
AEM Publish (production)	<code>https://<publish-host>/content/_cq_graphql/global/endpoint.json</code>

All requests are HTTP POST with `Content-Type: application/json`:

```
{
  "query": "{ becomeACustomerByPath(_path: \"...\") { item { heading } } }"
```

For persisted queries use HTTP GET:

GET /graphql/execute.json/btwholesale/become-a-customer-page

6.6 GraphQL Type Names

AEM generates type names from the model `jcr:title` — removes spaces, appends `Model`:

Model <code>jcr:title</code>	GraphQL Type	List query	ByPath query
Page Meta	PageMetaModel	pageMetaList	pageMetaByPath
Page Hero	PageHeroModel	pageHeroList	pageHeroByPath
Icon Card	IconCardModel	iconCardList	iconCardByPath
Nav Category	NavCategoryModel	navCategoryList	navCategoryByPath
Link Item	LinkItemModel	linkItemList	linkItemByPath
Contact Card	ContactCardModel	contactCardList	contactCardByPath
Portal Item	PortalItemModel	portalItemList	portalItemByPath
Become a Customer	BecomeACustomerModel	becomeACustomerList	becomeACustomerByPath
Help Content	HelpContentModel	helpContentList	helpContentByPath
Site Content	SiteContentModel	siteContentList	siteContentByPath

6.7 Example GraphQL Queries

Become a Customer — Full Page

```
{
  becomeACustomerByPath(
    _path: "/content/dam/btwholesale/content-fragments/become-a-customer/page"
  ) {
    item {
      meta {
        ... on PageMetaModel {
          title
          description
        }
      }
      hero {
        ... on PageHeroModel {
          breadcrumb
          breadcrumbHref
          heading
          subheading { html }
        }
      }
      benefitsHeading
      benefits {
        ... on IconCardModel {
          id
          icon
          heading
          description { html }
          highlight
        }
      }
    }
    productsHeading
    productsDescription { html }
    products {
```



```

portalsHeading
portals {
  ... on PortalItemModel {
    id icon heading description { html } label href
  }
}
}
}
}
}

```

Site Content — Header + Footer

```

{
  siteContentByPath(
    _path: "/content/dam/btwholesale/content-fragments/site-content/page"
  ) {
    item {
      logoAlt
      utilityLinks {
        ... on LinkItemModel { label href }
      }
      navLinks {
        ... on LinkItemModel { label href }
      }
      authLinks {
        ... on LinkItemModel { label href }
      }
      footerLegalLinks {
        ... on LinkItemModel { label href }
      }
      footerSocialLinks {
        ... on LinkItemModel { label href }
      }
      footerCopyright
    }
  }
}

```

List all Icon Cards (with pagination)

```

{
  iconCardList(
    _locale: "en"
    _limit: 10
    _offset: 0
  ) {
    items {
      _path
      id
      icon
      heading
      description { html }
    }
  }
}

```

Filter Nav Categories by ID

```

{
  navCategoryList(
    filter: {
      id: { _expressions: [{ value: "broadband" }] }
    }
  ) {
    items {
      id
      heading
      linkLabel
      linkHref
    }
  }
}

```

6.8 Persisted Queries

Persisted queries are named GraphQL queries stored in AEM. They: - Are served as cacheable HTTP GET requests (CDN-friendly) - Reduce query payload size on the client - Can be updated server-side without frontend deployments

Creating a persisted query (via AEM GraphQL or REST):

```

PUT /graphql/persist.json/btwholesale/become-a-customer-page
Content-Type: application/json

```

```

{ "query": "{ becomeACustomerByPath(...) { item { ... } } }" }

```

Executing a persisted query:

```

GET /graphql/execute.json/btwholesale/become-a-customer-page

```

With variables:

```

GET /graphql/execute.json/btwholesale/become-a-customer-page;path=/content/dam/...

```

6.9 Next.js / React Integration Pattern

```

// lib/aem.ts
const AEM_ENDPOINT =
  process.env.AEM_GRAPHQL_ENDPOINT ??
  'http://localhost:4502/content/_cq_graphql/global/endpoint.json';

async function aemQuery<T>(query: string): Promise<T> {
  const res = await fetch(AEM_ENDPOINT, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/json',
      // On Author: Basic auth. On Publish: no auth needed for published content.
      Authorization: `Basic ${Buffer.from('admin:admin').toString('base64')}`,
    },
    body: JSON.stringify({ query }),
    next: { revalidate: 60 }, // Next.js ISR - revalidate every 60s
  });
  const json = await res.json();
  if (json.errors) throw new Error(json.errors[0].message);
  return json.data as T;
}

```

```
// app/become-a-customer/page.tsx
import { aemQuery } from '@lib/aem';

const BECOME_A_CUSTOMER_QUERY = `{
  becomeACustomerByPath(
    _path: "/content/dam/btwholesale/content-fragments/become-a-customer/page"
  ) {
    item {
      hero { ... on PageHeroModel { heading subheading { html } } }
      benefitsHeading
      benefits { ... on IconCardModel { id icon heading description { html } } }
    }
  }
}`;

export default async function BecomeACustomerPage() {
  const data = await aemQuery<{ becomeACustomerByPath: { item: any } }>(
    BECOME_A_CUSTOMER_QUERY
  );
  const page = data.becomeACustomerByPath.item;
  return (
    <main>
      <h1>{page.hero.heading}</h1>
      <div dangerouslySetInnerHTML={ { __html: page.hero.subheading.html } } />
      <section>
        <h2>{page.benefitsHeading}</h2>
        {page.benefits.map((b: any) => (
          <div key={b.id}>
            <span className={ `icon icon-${b.icon}` } />
            <h3>{b.heading}</h3>
            <div dangerouslySetInnerHTML={ { __html: b.description.html } } />
          </div>
        ))}
      </section>
    </main>
  );
}
```

Using persisted queries (production recommended):

```
// Replaces the POST approach – GET is CDN cacheable
const res = await fetch(
  `${AEM_PUBLISH}/graphql/execute.json/btwholesale/become-a-customer-page`,
  { next: { revalidate: 300 } }
);
```

7. Inter-Fragment Reference Map

BecomeACustomerModel (/become-a-customer/page)

└─ meta	→	PageMetaModel	(/become-a-customer/meta)
└─ hero	→	PageHeroModel	(/become-a-customer/hero)
└─ benefits[]	→	IconCardModel	(/become-a-customer/benefits/*)
└─ products[]	→	NavCategoryModel	(/become-a-customer/products/*)
└─ whyBT[]	→	IconCardModel	(/become-a-customer/why-bt/*)
└─ applyCta	→	LinkItemModel	(/become-a-customer/apply-cta)

HelpContentModel (/help-content/page)

└─ meta	→	PageMetaModel	(/help-content/meta)
---------	---	---------------	----------------------

— hero	→	PageHeroModel	(/help-content/hero)
— categories[]	→	NavCategoryModel	(/help-content/categories/*)
— contactCards[]	→	ContactCardModel	(/help-content/contact-cards/*)
— viewAllContacts	→	LinkItemModel	(/help-content/view-all-contacts)
— portals[]	→	PortalItemModel	(/help-content/portals/*)

SiteContentModel (/site-content/page)

— meta	→	PageMetaModel	(/site-content/meta)
— utilityLinks[]	→	LinkItemModel	(/site-content/links/utility-*)
— navLinks[]	→	LinkItemModel	(/site-content/links/nav-*)
— authLinks[]	→	LinkItemModel	(/site-content/links/auth-*)
— productCards[]	→	IconCardModel	(/site-content/product-cards/*)
— supportActions[]	→	LinkItemModel	(/site-content/support-actions/*)
— whyCards[]	→	IconCardModel	(/site-content/why-cards/*)
— footerLegalLinks[]	→	LinkItemModel	(/site-content/footer/legal-*)
— footerSocialLinks[]	→	LinkItemModel	(/site-content/footer/social-*)

8. Deployment

Build and Deploy Commands

Action	Command
Build all modules	mvn clean install
Deploy all to local AEM	mvn clean install -PautoInstallSinglePackage
Deploy ui.content only	mvn -B clean install -PautoInstallPackage -pl ui.content
Deploy ui.apps only	mvn -B clean install -PautoInstallPackage -pl ui.apps
Deploy to publish (4503)	mvn clean install -PautoInstallSinglePackage -Daem.port=4503

FileVault Filter

```
<workspaceFilter version="1.0">
  <filter root="/conf/btwholesale" mode="merge"/>
  <filter root="/content/dam/btwholesale/content-fragments" mode="merge"/>
</workspaceFilter>
```

mode="merge" preserves nodes in AEM that are not in the package — safe for environments with additional authored content.

Deployment Order (clean instance)

1. ui.apps — registers Sling component resource types
2. ui.content — installs CF models, then CF data (FileVault handles ordering within the package)

9. Design Decisions and Constraints

Models must be flat (not in sub-folders)

The AEM CF Models console (/libs/dam/cfm/models/console/content/models.html/conf/btwholesale) only walks one level deep. Models in a shared/ sub-folder are not visible. All 10 models are kept directly under models/.

sling:OrderedFolder not dam:AssetFolder

AEM 2026.5 does not register dam:AssetFolder as a net-new creatable node type. All CF folder .content.xml files use jcr:primaryType="sling:OrderedFolder". Existing nodes already carrying dam:AssetFolder survive but cannot be newly created via package import.

No mix:versionable or jcr:isCheckedOut in packages

jcr:isCheckedOut is a protected JCR property controlled exclusively by the VersionManager. Including it with mode="merge" causes AEM to reject the node import. Neither property should appear in source-controlled .content.xml files.

cq:model belongs on data node, not jcr:content

AEM 2026.x resolves the CF model reference from jcr:content/data/@cq:model. Legacy documentation places it directly on jcr:content — this no longer works for GraphQL schema resolution.

Field name alignment is strict

The CF model field name attribute is the JCR property name written to the master variation on save. No alias layer exists. A mismatch silently returns null from GraphQL.

text-multi requires matching cfm-element

Rich text fields using the multieditor resource type must have cfm-element set to the same value as name. Without it, the editor cannot locate its rendering container and the field fails to persist.

10. Folder Structure Reference

CF Models

```
conf/btwholesale/settings/dam/cfm/models/
├── become-a-customer/      .content.xml
├── contact-card/           .content.xml
├── help-content/           .content.xml
├── icon-card/              .content.xml
├── link-item/              .content.xml
├── nav-category/           .content.xml
├── page-hero/              .content.xml
├── page-meta/              .content.xml
├── portal-item/            .content.xml
└── site-content/          .content.xml
```

CF Data

```
content/dam/btwholesale/content-fragments/
├── become-a-customer/
│   ├── page, meta, hero, apply-cta
│   └── benefits/      quality network revenue support
```

- products/ data voice hosted-comms managed-services mobile iot
- why-bt/ innovation reach bespoke
- help-content/
 - page, meta, hero, view-all-contacts
 - categories/ broadband ethernet hosted-communications
product-documentation customer-service-plans
pricing billing apps network-information
regulatory contracts faqs mobile
 - contact-cards/ enquiry callback phone
 - portals/ my-bt-wholesale the-hub
- site-content/
 - page, meta
 - footer/ partner-cta legal-* social-*
 - links/ nav-* auth-* utility-* hero-cta
 - product-cards/ data-connectivity hosted-communications
teams-phone-mobile voice-services
 - support-actions/ call contact help
 - why-cards/ expertise innovation reach